Práctica 2 · Flujos de entrada y salida y sockets TCP: el servidor de eco monohilo

Tiempo estimado: 2 h · Entrega voluntaria: código y respuestas breves, por Moodle · curso 2026-27

Objetivo

Al terminar la práctica sabrás abrir una conexión TCP desde un cliente y desde un servidor, montar sobre ella un lector de líneas y un escritor de líneas con autovaciado (ingl. *autoflush*), y escribir el bucle de eco que termina bien cuando el otro extremo cierra.

Material: transparencias «Entrada y salida en Java» y «Sockets en Java» (Moodle) · IntelliJ IDEA, Visual Studio Code o Eclipse · JDK 17 o posterior.

Referencias de la API de Java 21: Socket · ServerSocket.

Qué hay que hacer

El **cliente** lee una línea del teclado y la envía por el socket con `BufferedReader` y `PrintWriter`. Después espera la línea que el servidor le devuelve y la escribe por pantalla. El cliente termina cuando el usuario teclea el fin de fichero: en un terminal, Ctrl+D en Linux y macOS, y en Windows Ctrl+Z al principio de una línea seguido de Enter; en la consola de IntelliJ IDEA, Ctrl+D (Cmd+D en macOS).

El **servidor de eco** se ejecuta en `localhost`, puerto 9999. Acepta la conexión de un solo cliente; después lee cada línea del socket y se la devuelve, hasta que el cliente cierra la conexión.

Sobre el fin de fichero. El fin de fichero no viaja nunca por el socket. Cuando el cliente lo alcanza, sale de su bucle y cierra el socket; en el servidor, `readLine()` devuelve entonces `null`, que es la forma que tiene el final de la conversación en un protocolo de texto. No es un error, y no hay que devolver nunca una línea con la palabra `null`.

Sobre el autovaciado. Construye los dos `PrintWriter` con `true` como segundo argumento. Sin eso, cada línea se queda en el búfer local y los dos programas se quedan esperándose: el cliente espera un eco que el servidor ya ha escrito y que no ha salido de su máquina.

Completa los dos esqueletos que siguen. Los comentarios numerados marcan lo que falta.

Esqueleto del cliente

```
public class ClienteEcoTCP {

    public static final String MAQUINA = "localhost";
    public static final int PUERTO = 9999;

    public static void main(String[] args) {
        try (
             // 1. Un socket conectado a MAQUINA:PUERTO
             // 2. Un PrintWriter sobre el flujo de salida del socket, con autovaciado
             // 3. Un BufferedReader sobre el flujo de entrada del socket
             // 4. Un BufferedReader sobre System.in, para leer del teclado
            ) {

            // 5. Lee del teclado hasta el fin de fichero (readLine() == null)...
            // 6. ...enviando cada línea al servidor e imprimiendo el eco que llega

        } catch (IOException e) {
            System.err.println(e.getMessage());
            System.exit(1);
        }
        // al cerrarse el socket, el servidor ve el final de la conversación
    }
}
```

Esqueleto del servidor

```
public class ServidorEcoMonohilo {

    public static final int PUERTO = 9999;

    public static void main(String[] args) {
        try (/* 1. Un ServerSocket escuchando en PUERTO */) {

            try (
                 // 2. Espera (bloqueado en accept) a que entre un cliente
                 // 3. Un BufferedReader sobre el flujo de entrada del socket
                 // 4. Un PrintWriter sobre el flujo de salida, con autovaciado
                ) {

                // 5. Lee líneas del socket hasta que el cliente cierre (null)...
                // 6. ...devolviendo cada una tal cual

            }
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Comprobaciones y preguntas

1. Arranca primero el servidor y después el cliente, en dos terminales o en dos configuraciones de ejecución del entorno. Escribe tres líneas y comprueba que vuelven. Termina con el fin de fichero y comprueba que los dos programas terminan solos.
2. Quita el `true` del `PrintWriter` del servidor y vuelve a probar. ¿Qué ves, y en qué extremo? ¿Por qué no aparece ningún mensaje de error? Los dos programas se quedan parados y ya no responden al fin de fichero: detén el cliente (Ctrl+C en un terminal; en IntelliJ, el botón Stop, el cuadrado rojo) y verás que el servidor, al ver la conexión cerrada, termina solo. Deshaz el cambio después.
3. Haz que el servidor devuelva también el `null`. Sustituye el bucle de eco entero, es decir, la lectura que lo precede, el `while` y lo que hubiera después de él, por esta sola línea: `while (true) { String linea = entrada.readLine(); salida.println(linea); }`. ¿Qué imprime el cliente? ¿Y qué hace el servidor? Míralo en el administrador de tareas o el monitor del sistema. Deshaz el cambio después.
4. Con el servidor parado, arranca solo el cliente. ¿Qué excepción sale, y en qué línea? El manejador del esqueleto solo imprime el mensaje, así que para ver la clase de la excepción y dónde se ha lanzado cambia esa línea por `e.printStackTrace();`, o detén el programa en el depurador. Explica qué ha ocurrido en la red, y deshaz el cambio después.
5. Con el servidor en marcha y un cliente conectado, arranca un segundo cliente y escribe una línea. ¿Se conecta? ¿Recibe el eco? Explica qué está haciendo el único hilo del servidor en ese momento. (La práctica 3 resuelve esto.)

Entrega

Entrega voluntaria, sin nota. Si quieres que te la corrija, sube a la tarea de Moodle los dos ficheros `.java` (o un zip del proyecto) y las respuestas breves a las preguntas 2 a 5, de dos o tres frases cada una. La solución se publica después de la fecha de entrega.

Nota de la edición 2026-27

Mismo ejercicio que en cursos anteriores, con correcciones y bajo la numeración de este curso (era la práctica 4.1). En la hoja de 2021 el esqueleto estaba etiquetado como «solución»; dos comentarios confundían el flujo de salida con el de entrada; el comportamiento del fin de fichero no se decía en ninguna parte, y el entregable era un PDF en lugar de código. Los identificadores de esta edición están en castellano; la edición en inglés conserva los suyos, y cada una tiene su propia solución.